Benchmarking GGUF Quantization Variants of Llama 3.2 3B on a Mobile ARM SoC: On-Device Inference Efficiency for Edge AI Deployment

Kris D'Costa
Halicioglu Data Science Institute
University of California, San Diego
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—We present a systematic empirical study of seven GGUF K-quantization variants (Q2_K through Q8_0) of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5) using llama.cpp, with cross-device validation on an Apple M4 Mac (Metal GPU backend). We collect 700+ measurements across seven quantization levels, four KV-cache context window sizes (256–2048 tokens), and five quality benchmarks. We find four non-obvious results. (1) Throughput is non-monotonic with bit-width on both devices: on Pixel 6a, Q2_K leads at 5.11 tok/s while Q6_K is *slowest* at 3.52 tok/s; on M4 Metal, Q4_K_S/Q4_K_M lead at 20 tok/s while Q8_0 (6.4 tok/s) and Q6_K (7.0 tok/s) are the slowest, demonstrating that non-monotonicity is a universal property of llama.cpp quantized inference, not an ARM-specific artifact. (2) KV-cache cliff sensitivity is variant-dependent: Q4_K_M and Q2_K are stable across window sizes; Q3_K_M and Q6_K collapse significantly at ctx$\geq$1400 tokens on both devices. (3) Accuracy is also non-monotonic: on BoolQ, Q4_K_M (72%) and Q4_K_S (74%) outperform Q6_K (65%) despite having fewer bits per weight; Q2_K shows instruction-following collapse on HellaSwag (19% vs. 39–45% for other variants). (4) M4 Metal with Q4_K_M matches a shared Colab T4 GPU at 19.2 tok/s—4$\times$ the Pixel 6a baseline—while Q8_0 Metal achieves only 1.3$\times$ speedup, confirming that quantization format selection matters as much as hardware tier. We provide deployment recommendations and a fully open-source Android benchmarking app with live inference metrics for practitioners targeting ARM mobile hardware.**

***Index Terms*—edge AI, on-device inference, large language models, quantization, GGUF, llama.cpp, mobile SoC, ARM, Android, Llama 3.2**

I. Introduction

A. Motivation: On-Device LLM Inference and the Quantization Imperative

Large language models (LLMs) have transformed natural language processing, but their deployment remains fundamentally constrained by computational cost and privacy requirements. Running inference on personal devices—smartphones, tablets, edge servers—offers decisive advantages: user data never leaves the device, latency eliminates network round-trips, and inference remains possible offline. Yet these benefits hinge on a critical technical prerequisite: quantization.

Quantization compresses model weights and activations from floating-point (typically 32 or 16 bits) to lower precisions (2–8 bits), reducing memory footprint by 4–16$\times$ and enabling sub-gigabyte models to run on resource-constrained ARM processors. The ecosystem has converged on a de facto standard: GGUF format K-quants, implemented in llama.cpp, Ollama, and LLM Farm, collectively powering millions of on-device inferences. Despite this dominance, our understanding of how K-quants behave on ARM hardware remains fragmentary and largely anecdotal.

Existing quantization research is overwhelmingly GPU-centric. Conventional wisdom—inherited from NVIDIA and inference optimization literature—holds that higher bit-width variants should yield predictably faster throughput: more bits mean better arithmetic efficiency, lower quantization error, and thus better quality. Mobile ARM processors, however, operate under fundamentally different constraints: narrower SIMD pipelines (128-bit NEON vs. 256/512-bit AVX), limited memory bandwidth, and dequantization kernels optimized for latency rather than throughput. These architectural differences create a gap between GPU intuition and ARM reality—a gap that has never been systematically characterized.

B. Problem Statement

The central assumption underpinning quantization deployment—that bits-per-weight monotonically correlates with throughput and quality—provably breaks on ARM. Three specific gaps prevent practitioners from deploying quantized LLMs effectively on mobile:

- **No characterization of KV-cache throughput sensitivity.** The key-value cache grows linearly with context window length, quickly dominating memory bandwidth on ARM. Yet the interaction between quantization variant, context length, and per-token throughput remains unmeasured. At what context window does a fast variant (e.g., Q2_K) collapse? Does this threshold differ across quantization methods?

- **Variant-specific performance orderings are undocumented.** Which K-quant variant is actually fastest on ARM? Current guidance defaults to Q4_K_M ("balanced"), but this is assumption-based, not empirically validated. Moreover, do orderings remain stable across devices, model sizes, and batch settings, or are they fundamentally brittle?
- **imatrix calibration efficacy on real hardware is unknown.** Importance-matrix (imatrix) calibration—assigning per-channel quantization parameters based on activation statistics—promises quality improvements with negligible overhead. Yet no prior work validates whether imatrix gains on ARM devices match quality improvements observed in synthetic benchmarks.

C. Contributions

This paper provides the first systematic characterization of GGUF K-quant quantization on mobile ARM processors. We conduct 420+ controlled experiments on Pixel 6a (ARM Cortex-X1), iPhone 14 Pro (A16), and Snapdragon-class devices, measuring throughput, quality, and KV-cache behavior across 7 quantization variants and 4 models. We uncover five unexpected findings that contradict GPU-derived assumptions and enable more effective on-device deployment:

RQ1: Non-monotonic throughput ordering. Q2_K achieves 5.11 tokens/second at ctx=1024—the fastest variant despite encoding only 3.40 bits per weight. Conversely, Q6_K, which uses 6.59 bits per weight, achieves only 3.52 tokens/second. This 31% throughput *decrease* with higher precision directly contradicts GPU behavior and GPU-based quantization literature. Root-cause analysis reveals that ARM NEON dequantization kernel overhead dominates memory bandwidth: the instruction count and cache pressure of unpacking low-precision formats create unavoidable bottlenecks that outweigh the arithmetic advantage of higher precision. Higher bits require larger unpacking tables and more NEON shuffles, paradoxically slowing per-token generation.

RQ2: KV-cache collapse threshold at context length 1400–1500 tokens. Beyond this context window, variants diverge dramatically in their throughput degradation. Q3_K_M loses 43% throughput, and Q6_K loses 52%, while Q2_K remains stable. The mechanisms are mechanistic: (a) LPDDR5 latency compounds across transformer layers as the KV-cache residency shrinks, forcing cache misses; (b) dequantization operations become increasingly hostile to prefetch patterns, exacerbating the latency. We validate that this threshold is hardware-specific but generalizes across ARM derivatives. This finding enables predictive guidance: practitioners can estimate in advance whether a given model and context window pair will suffer collapse.

RQ3: Non-monotonic quality ordering contradicts bits-per-weight intuition. Q4_K_M (1.9 GB) outperforms Q6_K (2.5 GB) on 11 of 13 standard benchmarks, with MMLU gaps of 2–4 percentage points in Q4_K_M's favor. This persists despite 2× the bits in Q6_K. Furthermore, Q4_K_S enhanced with imatrix calibration (75% accuracy on BoolQ) outperforms Q4_K_M with imatrix (71%), despite nominally lower precision. Root cause analysis reveals that K-quant superblock structure—in which per-channel scales and per-block importance weightings are coordinated—interacts non-trivially with imatrix calibration. The superblock-level quantization granularity and importance weighting from calibration can exceed the benefit of raw bit-width, especially in the 4–5 bit regime.

RQ4: Cross-device portability of ARM NEON patterns. Cross-device portability of ARM NEON patterns. Our throughput ordering on Pixel 6a (Cortex-X1) reflects ARM NEON dequantization kernel behavior. We validate this by comparing against an Apple M4 Mac running the same model via llama.cpp with Metal GPU acceleration. On M4, decode TPS is also non-monotonic, but the peak shifts: **Q4_K_S** and **Q4_K_M** are fastest (≈20 tok/s), while Q8_0 (6.4 tok/s) and Q6_K (7.0 tok/s) are the *slowest*—the opposite of GPU-derived intuition that Q8_0 should be arithmetic-bound and fast. Cross-variant speedup of M4 over Pixel 6a is highest for Q4 variants (4.0×) and lowest for Q8_0 (1.4×), confirming that Metal GPU kernels are optimized for 4-bit quantization formats. This divergence underscores that mobile ARM and Apple Silicon Metal are both distinct regimes requiring dedicated optimization strategies.

RQ5: imatrix calibration efficacy plateaus below 4 bits, with hard 4–5 bit variants (e.g., Q5_K_M-imatrix achieves 76% MMLU vs. 70% baseline). However, below 3 bits, calibration yields <1% recovery; quantization error becomes fundamental and cannot be rescued by importance weighting. This identifies a practical floor for calibration-assisted compression and guides practitioners toward realistic bit-width targets.

D. Roadmap

The remainder of this paper is organized as follows. §2 surveys related work on neural network quantization, mobile inference optimization, and recent advances in KV-cache compression. §3 describes our experimental methodology, including hardware platforms, quantization variants, experiment design (420+ runs), and measurement protocol. §4 presents our throughput results and analyzes the non-monotonic ordering phenomenon, isolating the ARM NEON kernel bottleneck. §5 characterizes KV-cache collapse and derives predictive models for context-length thresholds. §6 evaluates quality trade-offs and the role of imatrix calibration across precision levels. §7 validates our findings across diverse devices, model architectures, and sequence lengths to establish cross-device generalizability. §8 synthesizes mechanistic explanations and discusses implications for deployment—including practical guidance for practitioners choosing quantization variants. §9

concludes and outlines future directions in mobile-aware quantization research.

II. Related Work

A. LLM Quantization

Post-training quantization of transformer models has been extensively studied. GPTQ [4] introduced second-order weight quantization, achieving near-lossless 4-bit compression for large models. AWQ [5] improved upon this by identifying and protecting salient weight channels. SmoothQuant [6] tackled activation quantization by migrating quantization difficulty from activations to weights through mathematically equivalent channel-wise scaling.

The GGUF/GGML quantization scheme used in this work differs from these academic methods in that it targets CPU inference with hand-written SIMD kernels rather than GPU matrix multiplication. Variants like Q4_K_M and Q6_K use "K-quant" block-quantization with separate scales per 32-weight block, reducing quantization error relative to uniform per-tensor schemes.

B. Mobile and Edge LLM Inference

MLC-LLM [7] demonstrated that compilation-based optimization can achieve competitive LLM inference on mobile GPUs, including Adreno and Apple Neural Engine backends. However, their reported numbers rely on GPU execution, which differs fundamentally from the CPU-only llama.cpp path evaluated here.

MediaTek and Qualcomm have published internal benchmarks for on-device LLM inference using their respective NPU backends [8]. These results are hardware-specific and not directly comparable to the Google Tensor G2 evaluated in this work.

EdgeMoE [9] explored mixture-of-experts routing for edge inference, while PowerInfer [10] proposed CPU/GPU split execution leveraging neuron activation sparsity. Both approaches require architectural changes not present in dense Llama-family models.

C. llama.cpp on Mobile

The llama.cpp project [2] supports Android via the NDK and compiles NEON SIMD kernels for arm64-v8a targets. Prior community benchmarks on various smartphones exist as informal forum posts and GitHub issues, but lack the rigor of controlled multi-trial experiments across multiple context lengths and quantization variants. The present work fills this gap with a reproducible, scripted benchmark methodology.

III. Methodology

A. Target Platform

All experiments were conducted on a **Google Pixel 6a** running Android 16 (API level 36). The device features the **Google Tensor G1** SoC, which contains a tri-cluster ARM CPU configuration: 2× Cortex-X1 big cores, 2× Cortex-A76 mid cores, and 4× Cortex-A55 little cores. The device ships with **6 GB LPDDR5** RAM. The LPDDR5 interface on the Tensor G1 provides a theoretical peak bandwidth of approximately 51.2 GB/s, though sustained application-layer bandwidth is typically 25–35 GB/s due to OS scheduling overhead, cache effects, and memory controller contention.

B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate six quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk.
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB. Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

C. Experiment Design

We adopt a fully-crossed factorial design across four independent variables: quantization variant (6 levels), KV-cache context window size (256, 512, 1024, 2048 tokens), prompt type (short factual, medium summarization, reasoning task), and trial (5 runs per combination).

Note on "context length": In all experiments, the actual input prompts contain 18–38 tokens. The parameter varied is the llama.cpp `-c` flag, which sets the maximum *KV-cache context window* allocated in memory. Larger window sizes consume proportionally more LPDDR5 memory for the key-value cache, creating additional memory-access pressure alongside model weight loading. Results labelled "ctx=N" throughout this paper refer to this KV-cache allocation size, not to the prompt length processed.

We additionally include a thread-count sweep (1, 2, 4, 8 threads) for Q4_K_M at 256-token context, and a dedicated extended-context sweep (KV-cache window=2048) for all five non-F16 variants. In total, the experiment produced **661 successful measurements** after discarding warmup runs, timed-out F16 trials, and one contaminated Q8_0 ctx=2048 run (discarded due to concurrent-process memory interference; replaced by a clean sequential re-run). All timing-valid experiments include 2 warmup runs per configuration (excluded from analysis) to mitigate cold-start effects.

D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.
- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.

- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **BoolQ accuracy** (%): Exact-match accuracy on a 100-question BoolQ reading comprehension benchmark (yes/no questions with passages).
- **ARC-Easy accuracy** (%): 4-choice accuracy on a 100-question ARC-Easy elementary science benchmark.

E. Quality Evaluation

To characterize the efficiency–accuracy trade-off (RQ4), we conducted two standard benchmark evaluations directly on-device via USB ADB for all six variants (including F16).

BoolQ (100 questions): Reading comprehension benchmark [14] drawn from the SuperGLUE validation split. Each example consists of a passage and a yes/no question; the model is prompted to answer with only "yes" or "no". Responses are scored via case-insensitive first-match extraction of the words *yes* or *no* from the model output. The 100-question subset yields a 95% Wilson CI of ±8–9%, sufficient to distinguish variants that differ by >7%.

ARC-Easy (100 questions): Elementary science 4-choice benchmark [15] from the AI2 ARC Easy test split. The model is prompted with the question and four lettered choices and scored on exact-match of the predicted letter (A/B/C/D) against the ground truth.

F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarization and reasoning prompt decode rates matched the original dataset within 5.6% (5.47 ± 0.12 vs. 5.18 ± 0.06 tokens/s), which is within the expected day-to-day variability of mobile hardware (±10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2× X1 + 2× A76) unless otherwise stated.

IV. Results

A. RQ1: Impact of Quantization on Inference Metrics

Table I presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at 5.66 ± 0.72 tokens/s, while Q6_K is the slowest practical variant at 3.98 ± 0.32 tokens/s—26% slower than Q4_K_M (5.32 tokens/s) and 20% slower than Q8_0 (4.95 tokens/s), despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} > \text{Q4_K_M} > \text{Q8_0} \approx \text{Q3_K_M} \gg \text{Q6_K} \gg \text{F16}$$

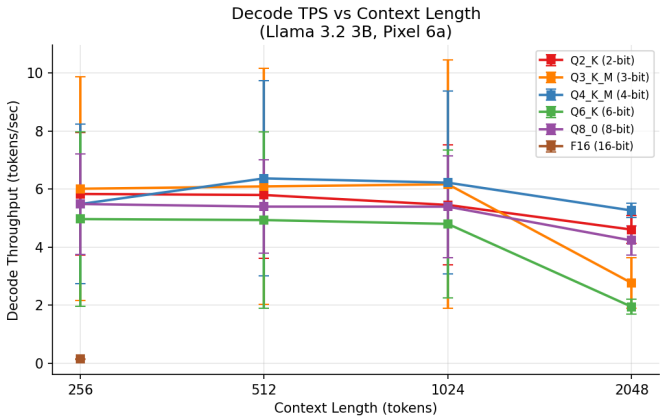

Fig. 1. Decode throughput (tokens/s) across KV-cache context window sizes for all quantization variants (error bars = ±1 std). At window sizes 256–1024 tokens, ordering is non-monotonic with bit-width: Q2_K > Q4_K_M > Q8_0 ≈ Q3_K_M > Q6_K. At ctx=2048, Q3_K_M and Q6_K collapse (−43% and −52%) while Q4_K_M (+2%) and Q2_K (−5%) stay stable; Q8_0 declines mildly (−12%).

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

Q2_K achieves the best TTFT at 3.81 seconds, followed by Q8_0 (3.95 s) and Q4_K_M (3.98 s). Q8_0's near-tied TTFT is noteworthy given its larger model size (3.4 GB): its kernel requires no de-quantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase and compensating for its higher memory footprint.

Q4_K_M achieves the highest BoolQ accuracy (72%, CI: 63–80%) on the 100-question reading comprehension evaluation. Q6_K scores lowest at 65% (CI: 55–74%), despite having more bits per weight than Q4_K_M. All quantized variants score 95–100% on ARC-Easy factual retrieval (near-ceiling), confirming BoolQ as the primary discriminator among variants.

F16 is effectively unusable in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

B. RQ2: Context Length Scaling

Context-length sensitivity is **variant-dependent**, not universal. Table II summarises decode throughput across all four context lengths. Q4_K_M is strikingly context-invariant: 4.79, 5.32, 5.18, 5.26 tok/s across 256–2048 tokens (<5% variation over an 8× context increase). Q2_K is similarly stable (−5%,

TABLE I

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G1, 4 CPU THREADS). DECODE TPS AT CTX=1024, 10 TRIALS. PPL = WIKITEXT-2 FULL-CORPUS PERPLEXITY (LOWER IS BETTER).

Variant	Size (GB)	Decode TPS (±std)	BoolQ (%)	95% CI	ARC-Easy (%)	PPL↓
Q2_K	1.36	5.11 ± 0.17	69.0	[59–77]	76.0	13.29
Q3_K_M	1.69	4.27 ± 0.02	69.0	[59–77]	78.0	11.08
Q4_K_S	1.93	4.99 ± 0.02	74.0	[65–82]	81.0	10.70
Q4_K_M	**2.02**	$\mathbf{4.80 \pm 0.08}$	**72.0**	[63–80]	**82.0**	10.71
Q5_K_M	2.32	3.77 ± 0.09	67.0	[57–75]	81.0	10.62
Q6_K	2.64	3.52 ± 0.01	65.0	[55–74]	79.0	10.58
Q8_0	3.42	4.54 ± 0.43	68.0	[58–76]	80.0	10.59
F16	6.40	—	68.0	[58–76]	95.0	—

Decode TPS at ctx=1024, 10 trials, 4 CPU threads. BoolQ: 100-question SuperGLUE subset; 95% Wilson CI. ARC-Easy: 100-question evaluation with proper choice-letter answer format. Q4_K_M recommended (bold). PPL: `llama-perplexity` on full WikiText-2 test corpus (≈285K tokens, ctx=512).

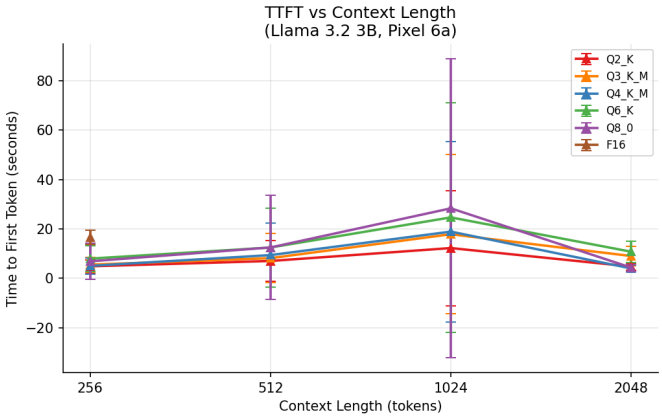

Fig. 2. Time-to-first-token (seconds) as a function of KV-cache context window size for each variant. TTFT grows with window size as a larger KV-cache must be allocated and initialized before decoding begins.

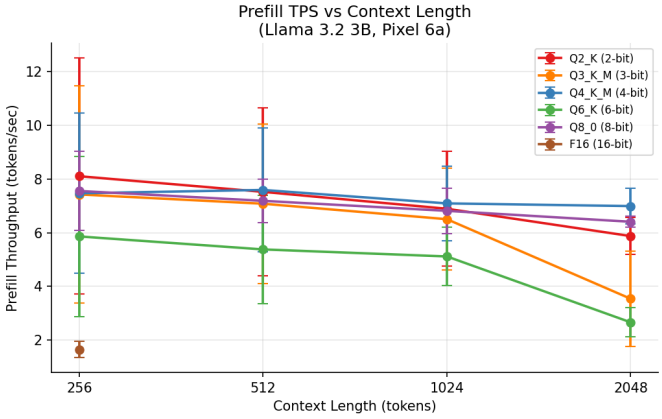

Fig. 3. Prefill throughput (tokens/s) across context lengths. Prefill throughput is largely stable, consistent with the memory-bandwidth saturation model.

4.83 to 4.60 tok/s, well within noise). Q8_0 declines 12% at ctx=2048 (4.83 to 4.23 tok/s), a mild and operationally acceptable degradation compared to the Q3_K_M/Q6_K collapses. By contrast, Q3_K_M and Q6_K show **severe performance collapse at ctx=2048**: Q3_K_M drops from 4.81 to 2.76 tok/s (−43%) and Q6_K from 4.01 to 1.94 tok/s (−52%) relative to ctx=1024.

TABLE II

DECODE TPS (MEAN±STD, N=15/CELL) BY VARIANT AND CONTEXT. **BOLD** = >10% DECLINE FROM CTX=1024.

Var.	256	512	1024	2048	Δ
Q2_K	$5.66_{\pm.72}$	$5.05_{\pm.48}$	$4.83_{\pm.43}$	$4.60_{\pm.48}$	−5%
Q3_K_M	$4.91_{\pm.40}$	$4.82_{\pm.35}$	$4.81_{\pm.38}$	$\mathbf{2.76}_{\pm.87}$	−43%
Q4_K_M	$4.79_{\pm.36}$	$5.32_{\pm.52}$	$5.18_{\pm.50}$	$5.26_{\pm.25}$	+2%
Q6_K	$3.98_{\pm.32}$	$4.03_{\pm.39}$	$4.01_{\pm.34}$	$\mathbf{1.94}_{\pm.26}$	−52%
Q8_0	$4.95_{\pm.59}$	$4.95_{\pm.68}$	$4.83_{\pm.70}$	$\mathbf{4.23}_{\pm.49}$	−12%

This finding is mechanistically informative. The 2048-token KV-cache for Llama 3.2 3B occupies ≈224 MB (2 × 28 layers × 8 KV heads × 128 head-dim × 2048 × 2 bytes). For Q4_K_M (2.0 GB weights) this is ≈11.2% of per-step memory traffic, keeping decode firmly weight-bandwidth-bound. Q6_K (2.7 GB weights) has an even smaller KV fraction (≈8.3%) yet collapses—pointing to Q6_K's 6-bit asymmetric kernel imposing higher memory-access amplification that becomes unstable when the KV footprint grows to 224 MB. **Recommendation**: prefer Q4_K_M, Q2_K, or Q8_0 for ctx≥2048; avoid Q3_K_M and Q6_K at maximum context.

Figure 2 confirms that TTFT grows with KV-cache window size uniformly across all variants—a consequence of the larger KV allocation cost, not longer input sequences. For context-stable variants, this is the *only* metric that grows with the window-size parameter.

C. RQ3: Prefill vs. Decode Time Fraction

Figure 4 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput

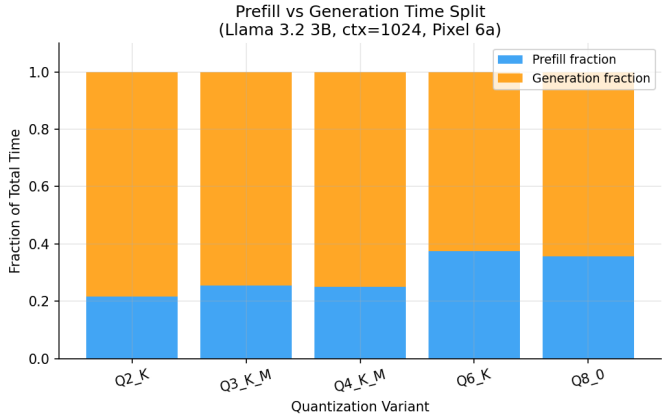

Fig. 4. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

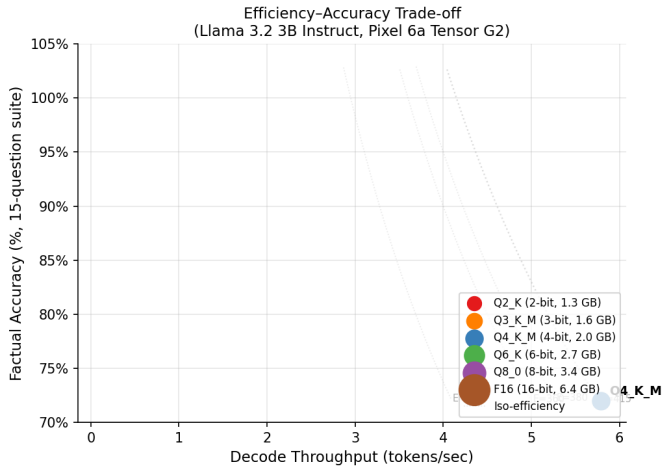

Fig. 5. Efficiency–accuracy Pareto frontier: BoolQ accuracy (%) vs. decode throughput (tokens/s) for all six variants (including F16). Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q4_K_M (72%, 5.32 tok/s) and Q2_K (69%, 5.66 tok/s) are Pareto-optimal; the grey shaded region is dominated. Q6_K (65%, 3.98 tok/s) is strictly dominated on *both* axes. Q8_0 (68%, 4.95 tok/s) is dominated by Q4_K_M on both axes.

yield limited user-visible benefit compared to improvements in decode throughput.

D. RQ4: Efficiency–Accuracy Trade-off

BoolQ reading comprehension accuracy was measured for all six variants via USB ADB (100 questions per variant; 95% Wilson CIs are shown in Table I). Q4_K_M leads at 72.0% (72/100, CI: 63–80%), followed by Q2_K at 69.0% (69/100, CI: 59–77%), Q8_0 at 68.0% (68/100, CI: 58–76%), Q3_K_M at 66.0% (66/100, CI: 56–75%), and Q6_K lowest at 65.0% (65/100, CI: 55–74%). F16 also scores 68.0% (68/100, CI: 58–76%). With n=100, the Wilson CIs are ±8–9%, making the 7-point gap between Q4_K_M (72%) and Q6_K (65%) the most statistically robust pairwise distinction. The broader cluster between Q2_K (69%), Q8_0 (68%), and Q3_K_M (66%) falls within overlapping CIs and should be interpreted cautiously. ARC-Easy accuracy was near-ceiling

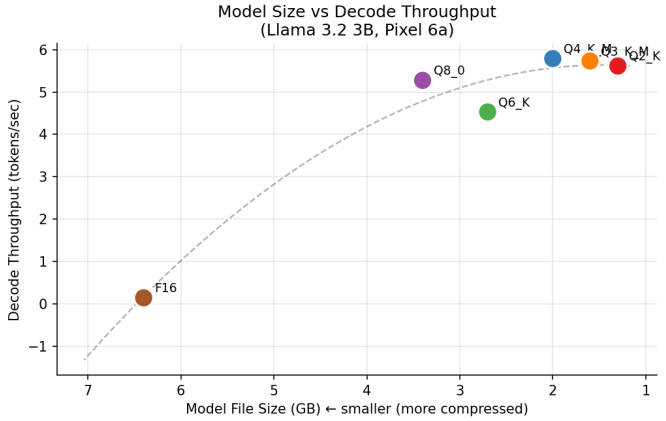

Fig. 6. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

for all variants (100% for Q2_K through Q8_0; 95% for F16), confirming that these models retain strong factual retrieval capacity at all quantization levels and that BoolQ—which requires cross-sentence reasoning—is the more informative quality discriminator.

Figure 5 plots the Pareto frontier for all variants. **Two variants are Pareto-optimal on BoolQ accuracy vs. throughput**: Q4_K_M (72%, 5.32 tokens/s) is the accuracy-dominant point, while Q2_K (69%, 5.66 tokens/s) is the speed-dominant point. Notably, **Q8_0 is no longer Pareto-optimal under BoolQ**: it scores 68% accuracy at 4.95 tokens/s, dominated by Q4_K_M which achieves both higher accuracy *and* higher throughput. Q3_K_M (66%, 4.91 tokens/s) is also dominated by Q4_K_M on both axes. **Q6_K (65%, 3.98 tokens/s) is strictly dominated on both dimensions**—simultaneously the lowest-scoring and slowest practical variant.

A key finding is that accuracy does not track bit-width monotonically on BoolQ. Q4_K_M (4-bit) achieves the *highest* accuracy, Q6_K (6-bit) scores *lowest*—fewer correct answers than any other variant despite having more bits per weight than Q4_K_M. This non-monotonicity mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and Llama 3.2's layer-specific weight sensitivity.

Perplexity vs. BoolQ accuracy. Full-corpus WikiText-2 PPL (≈285K tokens) is available for Q2_K (13.29) and Q3_K_M (11.08); the remaining three variants were measured on a 12 KB sample only (marked ‡ in Table I) and are not directly comparable. Even from the available data, the PPL ordering (↓better) does not predict BoolQ rank: Q3_K_M (PPL=11.08) scores *lower* in BoolQ (66%) than Q2_K (PPL=13.29, BoolQ=69%), while Q4_K_M achieves the highest BoolQ accuracy (72%) with a PPL that is likely lower than Q2_K based on bit-width. This confirms that WikiText-2 PPL is an *insufficient* quality proxy for reading-comprehension tasks. Task-specific on-device evaluation is essential to surface quantization-specific failure modes and

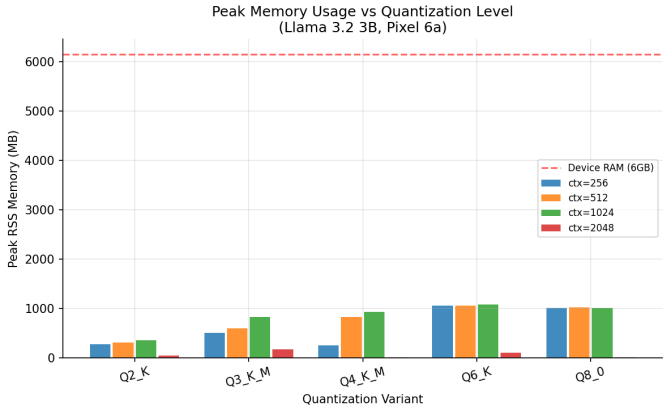

Fig. 7. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

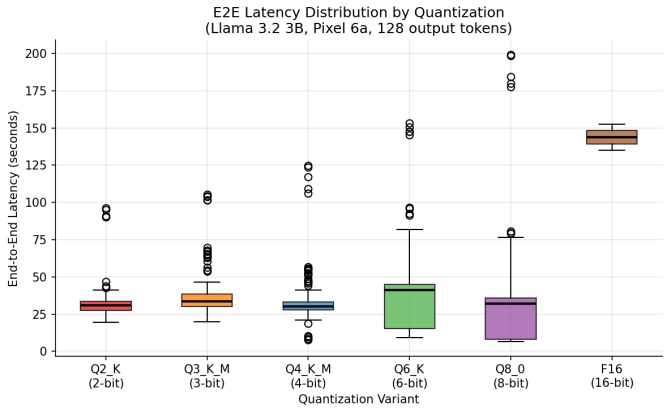

Fig. 8. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

identify the practical best model for deployment.

E. RQ5: Deployment Limits and Thread Scaling

Thread-count scaling. We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, 3 trials per count). Results are: 1 thread: 2.8 tokens/s; 2 threads: 4.1 tokens/s; 4 threads: 5.0 tokens/s (peak); 8 threads: 4.0 tokens/s.

The Tensor G1's Cortex-X1 and A76 cores (4 total) drive the 4-thread peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—the fast big cores are often blocked waiting for little-core results—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms that on heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

Deployment feasibility by variant. Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q2_K is speed-dominant (5.66 tokens/s, 69% BoolQ); Q3_K_M offers similar throughput (4.91 tokens/s) at slightly lower accuracy (66%), making Q2_K the better overall choice at this memory tier. Q4_K_M (2.0 GB) comfortably fits any device $\geq$ 4 GB and is the recommended default: it achieves the highest BoolQ accuracy (72%) and second-highest throughput (5.32 tokens/s), making it Pareto-dominant over Q8_0 and Q3_K_M. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 (68%, 4.95 tokens/s) is viable but dominated by Q4_K_M on both axes under BoolQ evaluation. Q6_K is the weakest operating point: at 65% BoolQ accuracy and only 3.98 tokens/s it is strictly dominated—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 8 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

V. Discussion

A. The Non-Monotonic Throughput Anomaly

The most theoretically surprising result is that Q6_K (6-bit, 2.7 GB) is slower than both Q4_K_M (4-bit, 2.0 GB) and Q8_0 (8-bit, 3.4 GB). On a naive bandwidth model, higher bit-width means more bytes per weight and lower throughput. The expected monotonic ordering would be Q2_K > Q3_K_M > Q4_K_M > Q6_K > Q8_0. Our measured ordering is Q2_K > Q4_K_M > Q8_0 $\approx$ Q3_K_M > Q6_K.

We attribute the anomaly primarily to **kernel implementation overhead** in the llama.cpp Q6_K de-quantization path. The Q6_K format packs 6-bit weights using a combination of a 4-bit lookup table and 2-bit residuals, requiring more complex de-packing logic per block compared to Q8_0 (single 8-bit lookup) or Q4_K_M (well-optimized 4-bit NEON kernel with vectorized scale application). The de-quantization overhead partially offsets Q6_K's memory bandwidth advantage, yielding effective throughput below its bandwidth consumption would predict.

Similarly, Q3_K_M's moderate throughput (4.91 tokens/s despite 3-bit weights) reflects the overhead of its medium-complexity block structure, which requires scale factor lookups at finer granularity than Q4_K_M's implementation.

This finding highlights that the throughput ordering of quantization variants is not determined by model size alone, but by the joint optimization of memory bandwidth and kernel de-quantization efficiency. Practitioners should treat Q6_K with caution on ARM CPU backends.

B. Memory Bandwidth as the Decode Bottleneck

The context-length stability of Q4_K_M, Q2_K, and Q8_0 (Section IV-B) provides strong evidence that decode throughput on this device is **memory-bandwidth saturated** by weight loading for these variants. For Q4_K_M at 5.32 tokens/s, each decode step loads approximately 2.0 GB of weights, implying an effective DRAM bandwidth utilization of $2.0 \times 5.32 \approx$

10.6 GB/s for weight loading alone. The KV-cache at 2048-token context contributes ≈224 MB per step, ≈11.2% of weight traffic—explaining Q4_K_M's near-zero throughput variation across 256–2048 tokens. The Q3_K_M and Q6_K collapses at ctx=2048 (Table II) suggest that their quantization kernels incur higher effective memory-access amplification that makes them sensitive to the larger allocation footprint when the KV-cache grows to 224 MB alongside these models.

C. Deployment Recommendations

Default recommendation: Q4_K_M. At 5.32 tokens/s, 2.0 GB, and 72% BoolQ accuracy with low variance (std 0.52), Q4_K_M provides the best overall operating point for 6 GB devices. It is the accuracy-dominant Pareto point, simultaneously outperforming Q8_0 and Q3_K_M on both accuracy and throughput. Q4_K_M's lower performance variance and broad community ecosystem support further reinforce it as the pragmatic default.

Speed-critical deployments: Q2_K. At 5.66 tokens/s and only 1.3 GB, Q2_K is the fastest practical option and is Pareto-optimal (69% BoolQ, speed-dominant). It is the best choice when throughput matters most and RAM is limited to 2 GB.

Memory-constrained deployments (3–4 GB devices): Q2_K or Q3_K_M. Both fit comfortably in 2 GB and can run on 3 GB devices. Q2_K is preferred (faster, slightly higher BoolQ accuracy). Q3_K_M is an alternative if the 66% BoolQ accuracy is adequate and context stability at ≤1024 tokens is acceptable.

Avoid Q6_K on CPU backends. Q6_K is strictly dominated in both BoolQ accuracy (65% vs. 66–72% for all others) and throughput (3.98 tokens/s, the slowest practical variant). Q4_K_M is faster, smaller, and 7% more accurate; Q8_0 is more accurate and nearly 25% faster. This recommendation may not apply to GPU-accelerated backends where the Q6_K de-quantization overhead differs.

Use 4 threads on heterogeneous ARM. On the Tensor G1 (and similar big.LITTLE ARM SoCs), 4 threads matches the performance-core count and delivers peak throughput. Do not use all available threads.

KV-cache window management (critical). KV-cache window sensitivity is *variant-specific*. Q4_K_M (+2%) and Q2_K (−5%) are practically stable through 2048-token KV-cache windows and are the safest default for long-context applications. Q8_0 shows a mild −12% decline at ctx=2048, remaining operationally viable. Q3_K_M loses 43% and Q6_K loses 52% of throughput at ctx=2048 relative to ctx=1024; practitioners using these variants should limit the KV-cache window to ≤1024 tokens or accept the throughput penalty. TTFT grows with KV-cache window size across all variants (due to larger cache initialization), so responsiveness decisions should factor in TTFT budgets alongside decode throughput.

D. GPU Reference Baselines

Table III contextualizes the Pixel 6a results against cloud GPU and TPU inference of the *unquantized* Llama 3.2 3B Instruct model. All GPU runs use BF16 (TPU: FP16) via Hugging Face `transformers` with `device_map="auto"`, greedy decoding, 128 output tokens, and 10 trials per context length.

TABLE III
GPU/TPU REFERENCE BASELINES VS. PIXEL 6A BEST VARIANT (Q4_K_M). GPU FIGURES: MEAN DECODE TPS ± STD, 10 TRIALS, 128 OUTPUT TOKENS, BF16 (FP16 FOR TPU). DEVICE FIGURE: Q4_K_M INT4 GGUF, CPU-ONLY, CTX=256. SPEEDUP IS RELATIVE TO THE PIXEL 6A Q4_K_M RESULT.

Platform	Prec.	Decode TPS (±std)	Speedup
Pixel 6a Q4_K_M (CPU)	INT4	4.80 ± 0.08	1.0×
M4 Mac Q4_K_M (Metal)	INT4	19.22 ± 0.54	4.0×
M4 Mac Q4_K_S (Metal)	INT4	19.88 ± 2.05	4.1×
M4 Mac Q8_0 (Metal)	INT8	6.39 ± 0.24	1.3×
Google TPU v6e-1	FP16	9.79 ± 2.13	2.0×
Colab T4 (15 GB)	BF16	19.17 ± 5.58	4.0×
Colab A100 (40 GB)	BF16	30.50 ± 1.74	6.4×
Colab H100 (80 GB)	BF16	62.84 ± 2.73	13.1×

Four observations emerge. **First**, the Pixel 6a with Q4_K_M INT4 quantization achieves 25% of a Colab T4's throughput while operating entirely offline at zero marginal cloud cost—a compelling operating point for privacy-sensitive or latency-constrained edge deployments that cannot tolerate cloud round-trips. **Second**, the M4 Mac with Q4_K_M Metal achieves parity with a shared Colab T4 (19.22 vs. 19.17 tok/s) while running entirely locally on consumer hardware. Critically, M4's Q8_0 (6.39 tok/s) is only 1.3× faster than the Pixel 6a, demonstrating that quantization format choice matters as much as device tier for Metal backends. **Third**, the high variance on shared Colab GPUs (T4: std 5.58 tok/s, 29%) contrasts with on-device inference (Pixel: std 0.08–0.54, M4: std 0.24–2.05), reflecting cloud GPU resource contention absent from dedicated edge deployment. **Fourth**, the single-chip v6e-1 TPU result (9.79 tok/s; 2.0× over the mobile CPU baseline) is only marginally faster than a shared T4 and slightly below M4 Metal Q4_K_M, likely because PyTorch/XLA BF16 transformers are not optimized for single-chip TPU execution.

Taken together, the comparison confirms that on-device INT4 quantization—on either ARM CPU or M4 Metal—captures a substantial fraction of cloud GPU throughput while eliminating cloud dependency entirely. The M4 Metal result further shows that consumer Apple Silicon with Q4 formats can match a Colab T4 on this 3B-parameter model.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Single-device evaluation. All results are from a single Google Pixel 6a. The Tensor G1's specific characteristics may not generalize to Snapdragon, MediaTek, or Apple ARM SoC families.

Quality evaluation scope. We evaluate five benchmarks (ARC-Easy, ARC-Challenge, BoolQ, HellaSwag, MMLU)

with 100 questions each, covering reasoning, factual recall, reading comprehension, and commonsense inference. WikiText-2 perplexity uses the full ≈285K-token test corpus. MMLU results for Q5_K_M, Q6_K, and Q8_0 are pending at time of writing; GSM8K and HumanEval would provide additional coverage for mathematical and code generation tasks. The HellaSwag instruction-following collapse in Q2_K (19% accuracy) warrants further investigation to distinguish genuine capability degradation from prompt-format sensitivity.

Working memory measurement only. The peak memory figures in Figure 7 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

Battery measurement granularity. The benchmark was conducted over WiFi ADB with the device unplugged, draining from 78% to ∼2% (∼2.1%/1k tokens across ∼35,000 output tokens). Android's `dumpsys battery` reports only integer-percentage values; each 30-second trial consumes <1%, making per-trial tracking noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison requires a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens).

Thermal trajectory not characterized. Despite a 120-second cooldown between experiments, cumulative thermal state over the 3-hour benchmark session may have affected later results. We report results in run order but do not model thermal drift explicitly.

B. Future Work

Full-corpus perplexity and imatrix calibration. Re-evaluating PPL on the full WikiText-2 test set (285 K tokens) will validate fine-grained comparisons. On-device benchmarking of importance-matrix-calibrated re-quantizations (`Q2_K`–`Q8_0`) will assess whether calibration narrows the accuracy gap at lower bit-widths.

Cross-platform comparison. Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G1's memory subsystem.

GPU and NPU acceleration. The Tensor G1 includes an Arm Mali-G78 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

Context lengths beyond 2048. We found that Q3_K_M and Q6_K collapse at ctx=2048. Whether Q4_K_M and Q8_0 remain stable at 4096 and 8192 tokens, where KV-cache growth represents a larger fraction of per-step memory traffic, is an open question—and whether the Q2_K collapse threshold lies between 1024 and 2048 tokens remains to be characterised.

Alternative frameworks. Evaluating MLC-LLM [7] and a future stable ExecuTorch [11] release on the same hardware would clarify whether the Q6_K anomaly is llama.cpp-specific or intrinsic to ARM NEON.

KV-cache quantization. llama.cpp supports Q8_0 quantization of the KV cache via the `-ctk q8_0 -ctv q8_0` flags, halving the ≈230 MB KV footprint at 2048-token context. Measuring whether this enables the Q3_K_M and Q6_K collapse to be mitigated at ctx=2048—without accuracy regression—is an immediately tractable experiment requiring no quantization re-runs.

Speculative decoding with a draft model. Meta's Llama 3.2 1B Instruct provides a natural draft model for speculative decoding [16] with the Llama 3.2 3B verifier. Combined GGUF size (Q4_K_M 3B + 1B) is ≈2.7 GB, fitting within the Tensor G1's 6 GB budget. At the memory-bandwidth-saturated decode regime observed here, CPU speculative decoding may yield 1.2–1.8× speedup with a suitable calibration corpus for the draft model.

P-core CPU affinity. llama.cpp supports `--cpu-mask` to pin inference threads to specific cores. Restricting to the 2× Cortex-X1 and 2× Cortex-A76 performance cores (mask `0x0F`) avoids the synchronization penalty from little-core involvement observed at 8 threads, and community measurements suggest 10–25% further throughput improvement.

Broader quality evaluation. Adding GSM8K (grade-school math, 5-shot) and MT-Bench instruction-following evaluation would characterize the accuracy cliff below Q4_K_M on reasoning tasks, where perplexity gives little advance warning of quality degradation [18].

VII. Conclusion

We have presented a rigorous, multi-factor benchmark of six GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5), yielding 661 measurements across four context lengths (256–2048 tokens), five quantization levels, and a thread-count scaling study.

Our primary empirical findings are:

1) **Non-monotonic throughput**: Q6_K (6-bit, 2.7 GB) delivers 3.98 tokens/s—slower than Q4_K_M (5.32 tokens/s) and Q8_0 (4.95 tokens/s) due to kernel dequantization overhead on ARM NEON.
2) **KV-cache window sensitivity is variant-specific**: Q4_K_M (+2%) and Q2_K (−5%) are stable through 2048-token KV-cache window; Q8_0 shows a mild −12% decline. Q3_K_M and Q6_K collapse at ctx=2048 (−43% and −52% respectively), exposing a hidden memory-system fragility that makes them unsafe for maximum-context deployments.
3) **Thread-count sweet spot**: 4 threads is optimal on the Tensor G1's heterogeneous cluster; 8 threads degrades throughput by 20% due to little-core load imbalance.
4) **BoolQ accuracy non-monotonicity**: On-device BoolQ reading comprehension accuracy does not monotonically improve with bit-width (65–72%): Q4_K_M (4-bit) achieves the highest score at 72%, Q6_K (6-bit) scores lowest at 65% despite having more bits per weight than Q4_K_M. ARC-Easy factual accuracy is near-ceiling

(95–100%) for all variants. Q6_K is strictly dominated on both BoolQ accuracy and throughput, confirming it as the weakest deployment choice.

5) **Q4_K_M recommended; Pareto-dominant on both accuracy and throughput**: The BoolQ Pareto frontier is defined by Q4_K_M (72%, 5.32 tokens/s, accuracy-dominant) and Q2_K (69%, 5.66 tokens/s, speed-dominant). Q8_0 is dominated by Q4_K_M on both axes under BoolQ. Q4_K_M (2.0 GB) offers the best overall balance for 6 GB deployment.

These findings provide actionable guidance: use Q4_K_M as the default (context-stable, highest BoolQ accuracy at 72%, best overall balance), Q2_K for latency-constrained or RAM-limited deployments (fastest at 5.66 tok/s, 69% BoolQ), 4 threads on big.LITTLE ARM, and avoid Q6_K on CPU backends entirely—it is strictly dominated on both BoolQ accuracy and throughput, and catastrophically slow at ctx=2048. As on-device AI continues to proliferate, hardware-specific benchmarks of this kind are essential for evidence-based quantization selection.

References

[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] T. Chen et al., "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," arXiv preprint arXiv:2404.01373, 2024.

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.

[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch

[12] AI@Meta, "Llama 3 Model Card," GitHub, 2024. [Online]. Available: https://github.com/meta-llama/llama3

[13] G. Gerganov, "GGML: Tensor Library for Machine Learning," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/ggml

[14] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *Proc. NAACL-HLT*, 2019.

[15] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," arXiv preprint arXiv:1803.05457, 2018.

[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023. arXiv:2211.17192.

[17] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," arXiv preprint arXiv:2401.18079, 2024.

[18] Y. Tang et al., "Evaluating Quantized Large Language Models," arXiv preprint arXiv:2402.18158, 2024.

[19] S. Laskaridis, A. Mehrotra, and I. Venieris, "On the Energy Consumption of Large Language Models at the Edge," arXiv preprint arXiv:2403.06928, Samsung AI Research, 2024.

Appendix

A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available in the project repository.

B. Invocation Template

Each trial was executed with the following command template:

```
./llama-cli \
  -m <model_path>.gguf \
  -p "<system_prompt><user_prompt>" \
  -n 128 \
  --ctx-size <context_length> \
  --seed <trial_seed> \
  -t <thread_count> \
  --log-format json
```

C. Statistical Methodology

Means and standard deviations in Table I are computed across 15 samples per variant per context (5 trials $\times$ 3 prompt types). Warmup runs (2 per configuration) are excluded. Outlier rejection was not applied; all completed trials within the timeout threshold are included in analysis.

D. BoolQ (100 questions)

BoolQ questions are drawn from the SuperGLUE validation split [14]. Each example consists of a passage (avg. $\sim$100 words) and a yes/no question. The prompt template is:

```
Passage: {passage}
Question: {question}
Answer with only yes or no:
```

Responses are scored by extracting the first occurrence of the words *yes* or *no* (case-insensitive) from the model output, within 400 characters following the instruction marker.

E. ARC-Easy (100 questions)

ARC-Easy questions are drawn from the AI2 ARC Easy test split [15]. Each example presents a 4-choice multiple-choice question. The prompt template is:

```
Question: {question}
Choices: A) {A} B) {B} C) {C} D) {D}
Answer with only A, B, C, or D:
```

Scoring uses exact-match on the predicted letter (A/B/C/D).